

Superform A-1

Security Audit

November 27, 2025

Version 1.0.0

Presented by [OxMacro](#)

Table of Contents

- [Introduction](#)
- [Overall Assessment](#)
- [Specification](#)
- [Source Code](#)
- [Issue Descriptions and Recommendations](#)
- [Security Levels Reference](#)
- [Issue Details](#)
- [Disclaimer](#)

Introduction

This document includes the results of the security audit for Superform's smart contract code as found in the section titled 'Source Code'. The security audit was performed by the Macro security team on November 10-27th 2025.

The purpose of this audit is to review the source code of certain Superform Solidity contracts, and provide feedback on the design, architecture, and quality of the source code with an emphasis on validating the correctness and security of the software in its entirety.

Disclaimer: While Macro's review is comprehensive and has surfaced some changes that should be made to the source code, this audit should not solely be relied upon for security, as no single audit is guaranteed to catch all possible bugs.

Overall Assessment

The following is an aggregation of issues found by the Macro Audit team:

Severity	Count	Acknowledged	Won't Do	Addressed
Medium	4	1	-	3
Low	4	2	-	2
Code Quality	15	-	-	15

Superform was quick to respond to these issues.

Specification

Our understanding of the specification was based on the following sources:

- Discussions with the Superform team.
- Available documentation in the repository.

Source Code

The following source code was reviewed during the audit:

- **Repository:** [v2-periphery](#)
- **Commit Hash:** 50a50fe31e4f661f6f339dc59d9eeaf1011db4bd

Specifically, we audited the following contracts within this repository.

Source Code	SHA256
src/Bank.sol	5aef0f5c866a86bbbed9337112302ec3453ebe0f3b6deba2eb9ca5cb678641107
src/SuperBank.sol	cb8ef4b611d8af50944c7bfc1d695911e514a5021b4ce30791f477786b6eccd3
src/SuperGovernor.sol	b9658c31555ceafdf4a640b2b1cf79c98099002a5e64be0e5e17082d99d2534c
src/SuperVault/SuperVault.sol	138c257dd1873d8fd0afeb8202a7faa9517437fce64cb3a152afe96f6697e26f
src/SuperVault/SuperVaultAggregator.sol	eb51620340329a4fbf1ed5dba4a0032499833373a821e0d6c9cecf36787c9da6
src/SuperVault/SuperVaultEscrow.sol	1334406f55194b0197e67640e6f084cc02982f856dcabad164079cf6e814b6a7
src/SuperVault/SuperVaultStrategy.sol	a5005f2b33a657a5e1b3c5aa88083212193effa936da94f9106eda683dc10ee3
src/UP/Up.sol	9d6cdb17c021c83abd5b3feaf33e2a706e5cdb3b55a3220cc3ad9ea407d97b3f
src/interfaces/IHookExecutionData.sol	8c10aa245d8376e9f67035e0999803d3

	e9a0b5aba0f486eb8475e095bf813e23
src/interfaces/ISuperBank.sol	3b1af18d8e0332b1247f592469dbab10 9d561b22e7276e65ee9b82aca51b494e
src/interfaces/ISuperGovernor.sol	f84c45a9fb3ac72617716b8ac24cfe34 4ba79017902607980b4eeb8ceb001379
src/interfaces/SuperVault/ISuperVault.sol	021873ce6182b29a91b6be1d78eb23ad 06f2de8e8de9d3826b5483a88aea57c3
src/interfaces/SuperVault/ISuperVaultAgg regator.sol	d43a152dd37f7da2aed10a5a8320f90a 695dcd83d51112243992c9dd5cb43924
src/interfaces/SuperVault/ISuperVaultEscr ow.sol	543d1c8b3580f081fcd98d6f870f06bb 0798754357f0f6ad2f2f6f6570465fce
src/interfaces/SuperVault/ISuperVaultStra tegy.sol	9aaee94c3a6053aaf2f8bf24ad4496c8 1e11aecb699695db8c935f853bafc577
src/interfaces/oracles/IECDSAPPSOracle. sol	61dd021df05fd4a4cbd90b304608f05d 89619f93b3dd7ecdcffe3b5c058c5e9a
src/interfaces/oracles/ISuperOracle.sol	da7bb85a8364127adc34ba2877032831 0e09ad65052c44cd41e86d7177ff5a0a
src/interfaces/oracles/ISuperOracleL2.sol	cd62cccc4b2e4c056ed6a811534792c5 85da28b297c7e71032d2b759fa00fb73
src/libraries/AssetMetadataLib.sol	18cf781c1807eb3ab7c59d65d95835f1 1e50faab5494e72c7f0210aa77c67d2a
src/libraries/SuperVaultAccountingLib.sol	18ff77e9d8d6127555b0774f836b6180 f3296b78a12ffb7d81cef10bd84d727f
src/oracles/ECDSAPPSOracle.sol	ddc96944bc82c463178829b75924613c 6b3fe7d9582b9f54ab43db70e4488f4c
src/oracles/SuperOracle.sol	5bfd2cf0ede99e518710ffe3ee8b31e1 0455f2fcdbee17506a05e0ad610ecd02

src/oracles/SuperOracleBase.sol	45df47d61618e37a91ad6fa5592196bb 302daf6314abcb6cd055a100d759d56b
src/oracles/SuperOracleL2.sol	73f6b7c40edaf8d4bca02c1476fd3d6c 4d64724aeb6ca7d446b629bf815e0699

Note: This document contains an audit solely of the Solidity contracts listed above. Specifically, the audit pertains only to the contracts themselves, and does not pertain to any other programs or scripts, including deployment scripts.

Issue Descriptions and Recommendations

Click on an issue to jump to it, or scroll down to see them all.

-  M-1 High water mark can be reset, allowing performance fees still be claimed even after poor performance
-  M-2 Redeem requests intending to be canceled can be fulfilled
-  M-3 `SuperOracleBase.cancelProviderRemoval()` is unusable due to missing caller implementation
-  M-4 Failure to replace old feeConfig when changing primary manager causes new manager to lose fees for 7 days
-  L-1 Inconsistent validation allows main manager to be a secondary manager during vault creation
-  L-2 Default slippage tolerance is set too high
-  L-3 Performance fee skimming can be DoS when fee amount is small relative to total supply
-  L-4 Manager can delay oracle updates
-  Q-1 Unused fields in `VaultCreationLocalVars` struct
-  Q-2 Unused struct fields in PPS forwarding logic
-  Q-3 Redundant condition in `isExempt` check reduces code clarity
-  Q-4 Redundant initiator field in `UpkeepWithdrawalRequest` struct wastes storage
-  Q-5 Redundant `isPaused` checks and suboptimal logic ordering
-  Q-6 Redundant Checks and Missing Invariant Enforcement for Strategy Update Intervals
-  Q-7 Remove unnecessary external call roundtrip in redeem and withdraw flows
-  Q-8 Unused external function `mintShares` increases contract size

- ~~Q-9~~ Unnecessary payable modifier in `fulfillRedeemRequests()`
- ~~Q-10~~ Redundant length check in `getQuoteFromProvider`
- ~~Q-11~~ Inefficient validator clearing process wastes gas by removing from index 0
- ~~Q-12~~ Redundant check in `_handleClaimRedeem()`
- ~~Q-13~~ Redundant check in `SuperVault.deposit()`
- ~~Q-14~~ Unused errors and events
- ~~Q-15~~ Nitpicks

Security Level Reference

We quantify issues in three parts:

1. The high/medium/low/spec-breaking **impact** of the issue:
 - How bad things can get (for a vulnerability)
 - The significance of an improvement (for a code quality issue)
 - The amount of gas saved (for a gas optimization)
2. The high/medium/low **likelihood** of the issue:
 - How likely is the issue to occur (for a vulnerability)
3. The overall critical/high/medium/low **severity** of the issue.

This third part – the severity level – is a summary of how much consideration the client should give to fixing the issue. We assign severity according to the table of guidelines below:

Severity	Description
(C-x) Critical	We recommend the client must fix the issue, no matter what, because not fixing would mean significant funds/assets WILL be lost.
(H-x) High	We recommend the client must address the issue, no matter what, because not fixing would be very bad, or some funds/assets will be lost, or the code's behavior is against the provided spec.
(M-x) Medium	We recommend the client to seriously consider fixing the issue, as the implications of not fixing the issue are severe enough to impact the project significantly, albeit not in an existential manner.
(L-x) Low	The risk is small, unlikely, or may not be relevant to the project in a meaningful way. Whether or not the project wants to develop a fix is up to the goals and needs of the project.
(Q-x) Code Quality	The issue identified does not pose any obvious risk, but fixing could improve overall code quality, on-chain composability, developer ergonomics, or even certain aspects of protocol design.
(I-x) Informational	Warnings and things to keep in mind when operating the protocol. No immediate action required.
(G-x) Gas Optimizations	The presented optimization suggestion would save an amount of gas significant enough, in our opinion, to be worth the development cost of implementing it.

Issue Details

M-1

High water mark can be reset, allowing performance fees still be claimed even after poor performance

TOPIC

Fee structure

STATUS

Fixed [↗](#)

IMPACT

Medium

LIKELIHOOD

High

The vault strategies primary manager is able to update the strategies fee config by first proposing new fee parameters via [proposeVaultFeeConfigUpdate\(\)](#) which allows this update to be set after a delay of 1 week. After this week has elapsed the primary manager can call [executeVaultFeeConfigUpdate\(\)](#) which sets the proposed values while also resetting the vault high water mark, used to determine performance, back to the current price per share:

```
// Get current PPS before updating fee config
uint256 currentPPS = getStoredPPS();
uint256 oldHwmPps = vaultHwmPps;

// Update fee config
feeConfig = proposedFeeConfig;
delete proposedFeeConfig;
feeConfigEffectiveTime = 0;

// Reset HWM PPS to current PPS to avoid incorrect fee calculations with n
vaultHwmPps = currentPPS;
```

Reference: [SuperVaultStrategy.sol#L513-523](#)

In cases where the vault has performed poorly, and the high water mark is higher

than the current price, it is beneficial for the manager to reset it back down to the current price in order to start earning performance fees again. Since there is no constraint on actually changing the fee configs on each proposal, a primary manager could have a similar config proposed before potentially volatile times allowing them to reset the high water mark in case of losses immediately and skip the expected week delay.

Additionally, if the vault is owed performance fees and `skimPerformanceFees()` is not called prior to updating the fee config, then the high water mark will be set to the current price without collecting owed performance fees.

Remediations to Consider

Rather than reset the high water mark, consider calling `skimPerformanceFees()` within the update before setting the new fee values, which allows fees to be calculated as expected if there are fees to collect. Additionally, consider instead of allowing the primary manager to reset the high water mark, give the super governor the ability to reset it in cases where a reset makes sense.

M-2 Redeem requests intending to be canceled can be fulfilled

TOPIC	STATUS	IMPACT	LIKELIHOOD
Underconstraint	Acknowledged	Medium	Medium

When a user calls `requestRedeem()` and then calls `requestCancelRedeem()` to cancel their redemption, the manager can still fulfill the original redemption request via `fulfillRedeemRequests()` instead of honoring the user's cancellation intention.

The root cause is in the `fulfillRedeemRequests()` function which processes pending redemptions without validating the cancellation flag. When iterating

through controllers, the function reads `pendingRedeemRequest` and proceeds to call `_processExactFulfillmentBatch()` without checking if `state.pendingCancelRedeemRequest == true`. Subsequently, within `_processExactFulfillmentBatch()`, it unconditionally sets `state.pendingCancelRedeemRequest = false`, effectively overriding the user's explicit cancellation request.

This creates a race condition where the manager can choose which request to fulfill first. If the manager fulfills the original redeem request despite the user's cancellation, the user's shares are redeemed against their explicit intention.

Remediations to Consider

In `fulfillRedeemRequests()`, add a check to skip for controllers that have `pendingCancelRedeemRequest == true`. This can be implemented by adding validation in the loop that processes controllers:

```
for (uint256 i; i < len; ++i) {
    if (i > 0 && controllers[i] <= controllers[i - 1]) revert CONTROLLERS
+    // Skip controllers with pending cancellation requests
+    if (superVaultState[controllers[i]].pendingCancelRedeemRequest) {
+        continue;
+    }
    ...
}
```

Reference: [src/SuperVault/SuperVaultStrategy.sol#L336-L350](https://github.com/Uniswap/super-vault/blob/master/contracts/SuperVaultStrategy.sol#L336-L350)

This ensures that users' cancellation requests are respected and prevents managers from unintentionally or maliciously fulfilling redemptions that users have explicitly requested to cancel.

RESPONSE BY SUPERFORM

It's intended behavior

Recommendation looks like equivalent to making the redeem request cancel instantaneous instead of a 2 steps process, which is what we had before already and we moved away from this design to avoid the user abusing this insta-cancel and trick the strategist into accumulating losses, while piling up assets into "free assets" to be able to eventually fulfill the redeem request, that would be eventually socialized.

In the 2 steps process design (current one) instead, the strategist can in certain cases decide to ignore the request to cancel a pending redeem request and fulfill it instead at a loss for the user

M-3

SuperOracleBase.cancelProviderRemoval() is unusable due to missing caller implementation

TOPIC	STATUS	IMPACT	LIKELIHOOD
Access Control	Fixed ↗	Medium	Medium

The `SuperOracleBase.cancelProviderRemoval()` function is designed to cancel a pending provider removal by deleting the `pendingRemoval` struct. However, this function is restricted to only be callable by the `SuperGovernor` contract.

The issue is that within the `SuperGovernor` contract, there is no implementation that calls this function. The `SuperGovernor` contract does not use hooks and is not upgradeable, meaning there is no mechanism to invoke `cancelProviderRemoval()`. This design flaw renders the `SuperOracleBase.cancelProviderRemoval()` function completely useless, as this core function can never be executed.

Remediations to Consider Add a corresponding function in the `SuperGovernor` contract to properly invoke `cancelProviderRemoval()`.

M-4

Failure to replace old feeConfig when changing primary manager causes new manager to lose fees for 7 days

TOPIC

Fee

STATUS

Fixed [↗](#)

IMPACT

High

LIKELIHOOD

Low

The `SuperVaultAggregator.changePrimaryManager()` function is used to immediately replace the main manager in an emergency situation. However, this function does not remove the `feeConfig` of the old main manager. If the emergency situation is related to a malicious/compromised main manager, the governor is not able to immediately change the fee recipient.

As a result, after calling this function, the old main manager's `feeRecipient` remains stored in the strategy and continues to collect `managementFee` when users deposit/mint. To change the `feeConfig`, a 7-day timelock must be waited out.

The new manager has to propose a new `feeConfig` and wait 7 days. During this 7-day period, the new manager loses:

- `managementFee`, as the fee recipient is still the old one
- `performanceFee`, as executing the new fee config updates the high water mark pps to the latest price

Remediations to Consider

Also allow changing the fee recipient immediately in

```
SuperVaultAggregator.changePrimaryManager()
```



Inconsistent validation allows main manager to be a secondary manager during vault creation

TOPIC

Access Control

STATUS

Fixed [↗](#)

IMPACT

Low

LIKELIHOOD

Low

In `SuperVaultAggregator` contract, the `createVault()` function lacks validation to prevent the main manager from also being included in the secondary managers list. The function sets `_strategyData[strategy].mainManager` and then iterates through `params.secondaryManagers` to add them, without checking for duplicates with the main manager.

This contradicts the protocol's design elsewhere: `addSecondaryManager()` explicitly reverts with `MANAGER_ALREADY_EXISTS()` [if attempting to add the primary manager as secondary](#), and `executeChangePrimaryManager()` [removes the new primary manager from the secondary managers set to maintain role separation](#).

This inconsistency allows creating vaults with overlapping manager roles, breaking an intended invariant and causing potential confusion in permission management.

Remediations to Consider

Add validation in `createVault()` when iterating through secondary managers, to not have the same address as the main manager.



Default slippage tolerance is set too high

TOPIC

Slippage Protection

STATUS

Fixed [↗](#)

IMPACT

Medium

LIKELIHOOD

Low

The contract defines a default slippage tolerance for redemptions in `DEFAULT_REDEEM_SLIPPAGE_BPS`. Currently, this constant is set to `100`. Since `BPS_PRECISION` is `10_000`, a value of `100` corresponds to a **1%** slippage tolerance.

For a yield-bearing vault, a 1% loss in principal during redemption is significant. If a strategy earns 5-10% APY, a 1% slippage loss represents approximately 1 to 2 months of yield. While users can override this value, unsophisticated users or integrations relying on the default value may unknowingly accept unfavorable execution prices during periods of volatility or low liquidity.

A safer default would be considerably lower, such as 0.1% (10 BPS), to protect user principal by default while still allowing for minor price movements.

Remediations to Consider

Decrease the default slippage tolerance to 0.1% (10 basis points) to provide better default protection for users.

```
- uint16 private constant DEFAULT_REDEEM_SLIPPAGE_BPS = 100;  
+ uint16 private constant DEFAULT_REDEEM_SLIPPAGE_BPS = 10;
```

Reference: [SuperVault/SuperVaultStrategy.sol#L51-L52](#)

L-3

Performance fee skimming can be DoS when fee amount is small relative to total supply

TOPIC	STATUS	IMPACT	LIKELIHOOD
DoS	Acknowledged	Low	Low

When a manager calls `SuperVaultStrategy.skimPerformanceFee()`, the transaction

can revert unexpectedly when the performance fee is small relative to the vault's total supply. This issue arises because the `ppsReduction` calculation employs floor rounding, which can result in a value of zero:

```
uint256 ppsReduction = Math.mulDiv(
    fee,
    PRECISION,
    totalSupplyLocal,
    Math.Rounding.Floor
);
```

Reference: [SuperVault/SuperVaultStrategy.sol#L437-L438](#)

If `fee < (totalSupplyLocal / PRECISION)`, `ppsReduction` rounds down to zero. Consequently, `newPPS` remains equal to `oldPPS`, triggering a revert in `updatePPSAfterSkim()` due to the requirement that the Price Per Share (PPS) must decrease after a skim:

```
// VALIDATION 1: PPS must decrease after fee skim
if (newPPS >= oldPPS) revert PPS_MUST_DECREASE_AFTER_SKIM();
```

Reference: [SuperVault/SuperVaultAggregator.sol#L300-L301](#)

Remediations to Consider

Consider setting `PRECISION` to `1e18` for if the asset decimals smaller than 18. This ensures sufficient granularity in PPS calculations, regardless of the underlying asset's decimals.

RESPONSE BY SUPERFORM

There should be no "unexpected revert" since the TX always gets offchain simulatedSecond, even though I agree their precision related point makes sense, it is useless for us to fix it since the amount of fee we would be able to collect is so small we won't collect it anyway because the gas cost could be

superiorTake their example: implementing the fix would allow us to claim 0.999 USDC in performance fee, on Mainnet likely this TX would cost us around 3 USD so ofc we won't make it anywayFinally, the overstating effect on the PPS of this kind of small unclaimed performance fee should be negligible because these are negligible fees, when they become non negligible is when the strategist has more incentives to call skim() to cash out the performance

L-4 Manager can delay oracle updates

TOPIC	STATUS	IMPACT	LIKELIHOOD
Stale pricing	Acknowledged	Medium	Medium

When a manager skims performance fees via [skimPerformanceFees\(\)](#), the `lastUpdateTimestamp` is updated alongside the new price per share accounting for the fees taken in [updatePPSAfterSkim\(\)](#). The `lastUpdateTimestamp` is updated when skimming fees to prevent a oracle update for the same timestamp that was skimmed to override the price per share while not considering the fees taken. This, however, results in postponing the next oracle update by at least the strategy's set `minUpdateInterval`. In the worst case, a skim frontruns a oracle update, causing the update to occur at least two `minUpdateInterval` from the last update, and resulting in a stale price. A malicious manager could respond to a large price deviation, and frontrun it if they are currently at a high water mark, and postpone this update, giving time for the manager and related parties to potentially pull shares out, before a pause/circuit breaker would have occurred.

Remediations to Consider

Instead of updating the `lastUpdateTimestamp` when skimming fees, add a value that is a timestamp or nonce which is set/incremented when performance fees are skimmed. Then if each oracle update includes the value of it upon submitting the

price, if it differs then the pricing is inaccurate, and should revert.

RESPONSE BY SUPERFORM

Part of trust assumptions.

Unused fields in `VaultCreationLocalVars` struct

TOPIC	STATUS	QUALITY IMPACT
Code Quality	Fixed ↗	Low

The `VaultCreationLocalVars` struct in `SuperVaultAggregator.createVault()` function declares three fields (`success` , `assetDecimals` , `underlyingDecimals`) that remain unused throughout the function. This results in unnecessary memory allocation during struct initialization, increasing gas costs for vault creation.

Remediations to Consider

Remove the unused fields from the struct definition:

```
struct VaultCreationLocalVars {
    uint256 currentNonce;
    bytes32 salt;
-   bool success;
-   uint8 assetDecimals;
-   uint8 underlyingDecimals;
    uint256 initialPPS;
}
```

TOPIC

Code Quality

STATUS

Fixed [↗](#)

QUALITY IMPACT

Low

The `ForwardPPSArgs` struct in `SuperVaultAggregator` contains three unused fields: `validatorSets`, `totalValidator`, and `updateAuthority`. Similarly, `PPSUpdateData` struct contains unused `validatorSet` and `totalValidators` fields. In `forwardPPS()`, values from `ForwardPPSArgs` are passed to `PPSUpdateData` during the `_forwardPPS()` call, but these validator-related and authority fields are never accessed or utilized in any subsequent logic.

Remediations to Consider

Remove the unused fields from both structs to reduce gas costs and improve code clarity:

```
struct ForwardPPSArgs {
    address[] strategies;
    uint256[] ppss;
-   uint256[] validatorSets;
-   uint256 totalValidator;
    uint256[] timestamps;
-   address updateAuthority;
}

struct PPSUpdateData {
    address strategy;
    bool isExempt;
    uint256 ppss;
-   uint256 validatorSet;
-   uint256 totalValidators;
    uint256 timestamp;
    uint256 upkeepCost;
}
```




Redundant condition in isExempt check reduces code clarity

TOPIC

Code quality

STATUS

Fixed [↗](#)

QUALITY IMPACT

Low

In `SuperVaultAggregator` contract, the `isExempt` field uses condition `(!paymentsEnabled) | (upkeepCost == 0)`. This is redundant because `upkeepCost` is initialized to 0 and only assigned a non-zero value when `paymentsEnabled` is true. When `paymentsEnabled` is false, `upkeepCost` remains 0 by definition. Therefore, checking `!paymentsEnabled` is unnecessary since it's already implied by `upkeepCost == 0`.

Remediations to Consider

Simplify the condition to remove redundancy:

```
_forwardPPS(  
  PPSUpdateData({  
    strategy: strategy,  
-    isExempt: (!paymentsEnabled) || (upkeepCost == 0),  
+    isExempt: upkeepCost == 0,  
    pps: args.ppss[i],  
    validatorSet: args.validatorSets[i],  
    totalValidators: args.totalValidator,  
    timestamp: ts,  
    upkeepCost: upkeepCost  
  })  
);
```



Redundant initiator field in UpkeepWithdrawalRequest struct wastes storage

TOPIC

Code quality

STATUS

Fixed 

QUALITY IMPACT

Low

The `initiator` field in `UpkeepWithdrawalRequest` struct is redundant. Since `proposeWithdrawUpkeep()` enforces that only the `mainManager` can create withdrawal requests, and both `changePrimaryManager()` and `executeChangePrimaryManager()` delete any pending upkeep withdrawals during manager transitions, the invariant `initiator == mainManager` always holds for active requests. The `initiator` field serves two purposes in `executeWithdrawUpkeep()`: validating request existence in `changePrimaryManager()`, `executeChangePrimaryManager()` and `executeWithdrawUpkeep()` functions and determining transfer recipient in `executeWithdrawUpkeep()`. Both can be accomplished alternatively - the existence check can use `effectiveTime != 0` instead, and the transfer can be sent to `_strategyData[strategy].mainManager`.

Remediations to Consider

Remove the `initiator` field from `UpkeepWithdrawalRequest` struct. Modify to check `request.effectiveTime != 0` for request existence and transfer to `_strategyData[strategy].mainManager` instead of `request.initiator`.



Redundant `isPaused` checks and suboptimal logic ordering

TOPIC

Code quality

STATUS

Fixed 

QUALITY IMPACT

Low

`SuperVaultAggregator._forwardPPS()` function contains redundant `isPaused` checks. The calling function `forwardPPS()` already filters out paused strategies before invoking `_forwardPPS()`, making the `!_strategyData[args.strategy].isPaused` checks inside the `_forwardPPS()` function redundant.

Remediations to Consider

Remove redundant checks and reorder logic for efficiency:

```
-      if ((checksFailed| args.pps == 0) && !_strategyData[args.strategy].
-          ...LOGIC1...
-      }
-      if (!args.isExempt) {
-          ...LOGIC2...
-      }
-      if (!checksFailed && args.pps > 0) {
-          ...LOGIC3...
-      }

+      if (!args.isExempt) {
+          ...LOGIC2...
+      }
+      if ((checksFailed || args.pps == 0) {
+          ...LOGIC1...
+      } else {
+          ...LOGIC3...
+      }
```

Reference: [SuperVault/SuperVaultAggregator.sol#L1209-L1251](#)



Redundant Checks and Missing Invariant Enforcement for Strategy Update Intervals

The `SuperVaultAggregator` contract manages strategy timing parameters but fails to consistently enforce the invariant `minUpdateInterval < maxStaleness`.

First, `createVault()` function lacks validation for this invariant, allowing the creation of dysfunctional vaults where PPS updates might be impossible or immediately stale. Consequently, `_forwardPPS()` function essentially mitigates this by redundantly calculating `Math.min(minUpdateInterval, maxStaleness)` for rate limiting, wasting gas on every update. Conversely, `executeMinUpdateIntervalChange()` function contains a dead check `if (newInterval >= maxStaleness)`; since `maxStaleness` is immutable after creation and the condition is strictly enforced during the proposal phase (`proposeMinUpdateIntervalChange()`), this check is unnecessary.

Remediations to Consider

Enforce the invariant at the source in `createVault()` function and remove redundant checks in `_forwardPPS()` and `executeMinUpdateIntervalChange()` to improve gas efficiency and code clarity.

```
function createVault(...) ... {
    // ...
    if (params.maxStaleness < SUPER_GOVERNOR.getMinStaleness()) {
        revert MAX_STALENESS_TOO_LOW();
    }
+   if (params.minUpdateInterval >= params.maxStaleness) {
+       revert INVALID_VAULT_PARAMS();
+   }
    // ...
}

function _forwardPPS(...) ... {
    // ...
-   uint256 minInterval = Math.min(
```

```

-         _strategyData[args.strategy].minUpdateInterval,
-         _strategyData[args.strategy].maxStaleness
-     );
+     uint256 minInterval = _strategyData[args.strategy].minUpdateInterval
    // ...
}

function executeMinUpdateIntervalChange(...) ... {
    // ...
-     if (newInterval >= _strategyData[strategy].maxStaleness) {
-         return;
-     }
    _strategyData[strategy].minUpdateInterval = newInterval;
}

```

Reference: [SuperVault/SuperVaultAggregator.sol#L137-L214](#) ,
[SuperVault/SuperVaultAggregator.sol#L1151-L1253](#) ,
[SuperVault/SuperVaultAggregator.sol#L833-L863](#)



Remove unnecessary external call roundtrip in redeem and withdraw flows

TOPIC

Redundancy

STATUS

Fixed [↗](#)

QUALITY IMPACT

Low

The current implementation of `redeem()` and `withdraw()` functions in `SuperVault` contract executes a suboptimal call flow: `Vault.redeem()` calls `Strategy.handleOperations7540()`, which calls back `Vault.extractAndSendAssets()`, which finally calls `Escrow.returnAssets()`. This creates an unnecessary circular dependency (`Vault -> Strategy -> Vault -> Escrow`) and increases gas consumption. The root cause is located in `SuperVaultStrategy` attempting to handle the asset transfer via a callback to the Vault, instead of letting the Vault handle the transfer directly after the strategy

updates its state. The highest impact is increased transaction costs for users during every redemption or withdrawal.

Remediations to Consider

Refactor the logic to remove the external call to `Vault.extractAndSendAssets()` from `Strategy.handleOperations7540()`. Instead, call `Escrow.returnAssets()` directly in `Vault.redeem()` and `Vault.withdraw()` after the strategy operation.

```
strategy.handleOperations7540(  
    ISuperVaultStrategy.Operation.ClaimRedeem,  
    controller,  
    receiver,  
    assets  
);  
  
+ ISuperVaultEscrow(escrow).returnAssets(receiver, assets);
```

Reference: [SuperVault/SuperVault.sol#L484-L547](#)



Unused external function mintShares increases contract size

TOPIC	STATUS	QUALITY IMPACT
Redundancy	Fixed ↗	Low

The function `mintShares()` in `SuperVault` contract is defined to allow the `strategy` to mint shares directly. However, strategy contract never calls this function.

Remediations to Consider

Remove the `mintShares()` function and its declaration to improve code cleanliness

and reduce deployment costs.

Unnecessary payable modifier in `fulfillRedeemRequests()`

TOPIC	STATUS	QUALITY IMPACT
Code Quality	Fixed ↗	Low

The function `fulfillRedeemRequests()` in `SuperVaultStrategy` contract is declared as `payable` despite performing no ETH operations. It strictly handles ERC20 transfers and accounting updates.

Remediations to Consider

Remove the `payable` modifier to strictly enforce non-payable execution.

Redundant length check in `getQuoteFromProvider`

TOPIC	STATUS	QUALITY IMPACT
Code quality	Fixed ↗	Low

The `activeProviders` array is strictly enforced to never exceed `MAX_SAMPLE_PROVIDERS` during provider addition in `_configureOracles`.

Consequently, the ternary check `length > MAX_SAMPLE_PROVIDERS` in `getQuoteFromProvider` is mathematically impossible to be true. This redundant operation adds unnecessary gas overhead and complexity.

Remediations to Consider

Assign `length` directly to `sampleCap` .

```
- uint256 sampleCap = length > MAX_SAMPLE_PROVIDERS ? MAX_SAMPLE_PROVIDERS  
+ uint256 sampleCap = length;
```

Q-11 Inefficient validator clearing process wastes gas by removing from index 0

TOPIC

Code quality

STATUS

Fixed [↗](#)

QUALITY IMPACT

Low

In the `SuperGovernor.setValidatorConfig()` function, the existing validators are cleared by repeatedly removing the element at index 0. This approach is gas-inefficient because `EnumerableSet.remove()` internally swaps the removed element with the last element before popping it. When removing from index 0, every removal operation requires this swap, resulting in unnecessary gas costs especially when the validator set is large.

```
uint256 oldLength = _validatorConfig.validators.length();  
for (uint256 i; i < oldLength; i++) {  
    _validatorConfig.validators.remove(_validatorConfig.validators.at(0));  
}
```

Reference: [src/SuperGovernor.sol#L399-L402](#)

Remediations to Consider

Use `clear()` function in the `EnumerableSet` library to remove all elements in a

single operation.

Q-12

Redundant check in `_handleClaimRedeem()`

TOPIC

Code quality

STATUS

Fixed [↗](#)

QUALITY IMPACT

Low

`SuperVaultStrategy._handleClaimRedeem()` function validates `state.maxWithdraw < assetsToClaim` but callers (`SuperVault.withdraw()` and `SuperVault.redeem()`) already enforce the same bound.

Remediations to Consider

Remove the redundant check and rely on caller validation.

Q-13

Redundant check in `SuperVault.deposit()`

TOPIC

Code quality

STATUS

Fixed [↗](#)

QUALITY IMPACT

Low

The `SuperVault.deposit()` function performs a redundant zero-amount validation after calling `strategy.handleOperations4626Deposit()`. The check `if (shares == 0) revert ZERO_AMOUNT()` is unnecessary since `handleOperations4626Deposit()` already validates and reverts for zero amounts internally.

Remediations to Consider

Remove the redundant `if (shares == 0) revert ZERO_AMOUNT()` check to reduce gas costs and simplify the code.

Q-14

Unused errors and events

TOPIC	STATUS	QUALITY IMPACT
Code quality	Fixed ↗	Low

Several error definitions and event declarations appear unused in the codebase.

Here is the list of those unused errors/events:

- `ISuperOracle.sol` : error `NOT_ADMIN()` , error `UNSUPPORTED_QUOTE()` , error `NO_PRICES()`
- `IECDSAPPSOracle.sol` : error `STRATEGY_MISMATCH()` , error `PPS_MISMATCH()` , error `INSUFFICIENT_VALIDATOR_PARTICIPATION()` , error `VALIDATOR_COUNT_MISMATCH()` , error `INVALID_VALIDATOR_SET()` , event `InsufficientGasForForward()`
- `ISuperVaultAggregator.sol` : event `UpkeepCostUpdated()` , event `GlobalHooksRootVetoed()` , event `StrategyHooksRootVetoed()` , event `StrategyUnpausePPSTimeLockUpdated()` , error `VAULT_PAUSED()` , error `UPDATE_TOO_STALE()` , error `INSUFFICIENT_UPKEEP_BALANCE()` , error `CANNOT_REMOVE_LAST_MANAGER()` , error `MANAGER_CHANGE_NOT_READY()` , error `NOT_A_GUARDIAN()` , error `NO_PENDING_ROOT_UPDATE()` , error `CANNOT_ADD_PROTECTED_KEEPER()` , error `UNPAUSE_TIMELOCK_NOT_MET()`
- `ISuperVaultStrategy.sol` : error `INSUFFICIENT_FUNDS()` , error

`INSUFFICIENT_SHARES()` , error `ALREADY_PROPOSED()` , error
`INVALID_REDEEM_FILL()` , error `SLIPPAGE_EXCEEDED()` , error
`ZERO_REQUEST_PPS()` , event `HookRootUpdated()` , event `HookRootProposed()` ,
event `FeePaid()` , event `VaultCostBasisUpdated()`

- `ISuperGovernor.sol` : error `VALIDATOR_NOT_REGISTERED()` , event `PPSOracleQuorumUpdated()`

Remediations to Consider

Remove unused error and event declarations or consolidate them in appropriate interfaces if intended for cross-contract use.

Nitpicks

TOPIC	STATUS	QUALITY IMPACT
Code quality	Fixed 	Low

1. In `SuperVault.requestRedeem()` , the code manually checks `if (owner != msg.sender && !isOperator[owner][msg.sender])` instead of reusing the internal `_validateController(owner)` function used elsewhere (e.g., `withdraw`, `redeem`).
2. The `SuperVaultAccountingLib.calculateAverageWithdrawPrice()` function uses a parameter named `currentAssetsWithFees`. This is misleading as there are no fees applied during withdrawal/redemption calculations in this context.
3. `SuperGovernor._convertGasToUp()` function has comments and variables refer to "ETH" when describing native token values (e.g., gas costs). For a multi-chain protocol deploying on chains like BNB Chain or Polygon, "NATIVE_TOKEN" would

be more accurate to avoid confusion.

4. The `SuperVault.initialize()` function does not emit an event, unlike other contracts (`SuperVaultStrategy` , `SuperVaultEscrow`).
5. In `Bank._executeHooks()`, `ISuperHook(hookAddress)` is cast multiple times redundantly, despite the typed `hook` variable already existing.
6. In `SuperVaultStrategy` contract, `theVaultHwmPps comment` claims it is only updated in `skimPerformanceFee` , incorrectly omitting its update in `executeVaultFeeConfigUpdate` .
7. In `SuperVaultStrategy` contract, the timelock duration of `1 weeks` is hardcoded as a magic number in `proposeVaultFeeConfigUpdate()` and `_proposePPSExpiration()` rather than using a named constant.
8. In `SuperVaultStrategy` contract, functions like `managePPSExpiration()` and `manageYieldSource()` use raw integers (0, 1, 2, 3) for actions instead of type-safe enums.
9. In `Bank._executeHooks()` function, the `HookValidationFailed` event is emitted immediately before a `revert HOOK_VALIDATION_FAILED()` . Since the revert undoes all state changes including event logs, this emission is useless.
10. The `_superformManagers` state variable in the `SuperGovernor` contract is declared but confirmed by the client as no longer being used in the current implementation.

Disclaimer

Macro makes no warranties, either express, implied, statutory, or otherwise, with respect to the services or deliverables provided in this report, and Macro specifically disclaims all implied warranties of merchantability, fitness for a particular purpose, noninfringement and those arising from a course of dealing, usage or trade with respect thereto, and all such warranties are hereby excluded to the fullest extent permitted by law.

Macro will not be liable for any lost profits, business, contracts, revenue, goodwill, production, anticipated savings, loss of data, or costs of procurement of substitute goods or services or for any claim or demand by any other party. In no event will Macro be liable for consequential, incidental, special, indirect, or exemplary damages arising out of this agreement or any work statement, however caused and (to the fullest extent permitted by law) under any theory of liability (including negligence), even if Macro has been advised of the possibility of such damages.

The scope of this report and review is limited to a review of only the code presented by the Superform team and only the source code Macro notes as being within the scope of Macro's review within this report. This report does not include an audit of the deployment scripts used to deploy the Solidity contracts in the repository corresponding to this audit. Specifically, for the avoidance of doubt, this report does not constitute investment advice, is not intended to be relied upon as investment advice, is not an endorsement of this project or team, and it is not a guarantee as to the absolute security of the project. In this report you may through hypertext or other computer links, gain access to websites operated by persons other than Macro. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such websites' owners. You agree that Macro is not responsible for the content or operation of such websites, and that Macro shall have no liability to your or any other person or entity for the use of third party websites. Macro assumes no responsibility for the use of third party software and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any outcome generated by such software.